

Konzeption eines Python-basierten Studien-Dashboards

Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)

Portfolio – Erarbeitungs-/Reflexionsphase / Phase 2

im Studiengang
B.Sc. Cyber Security

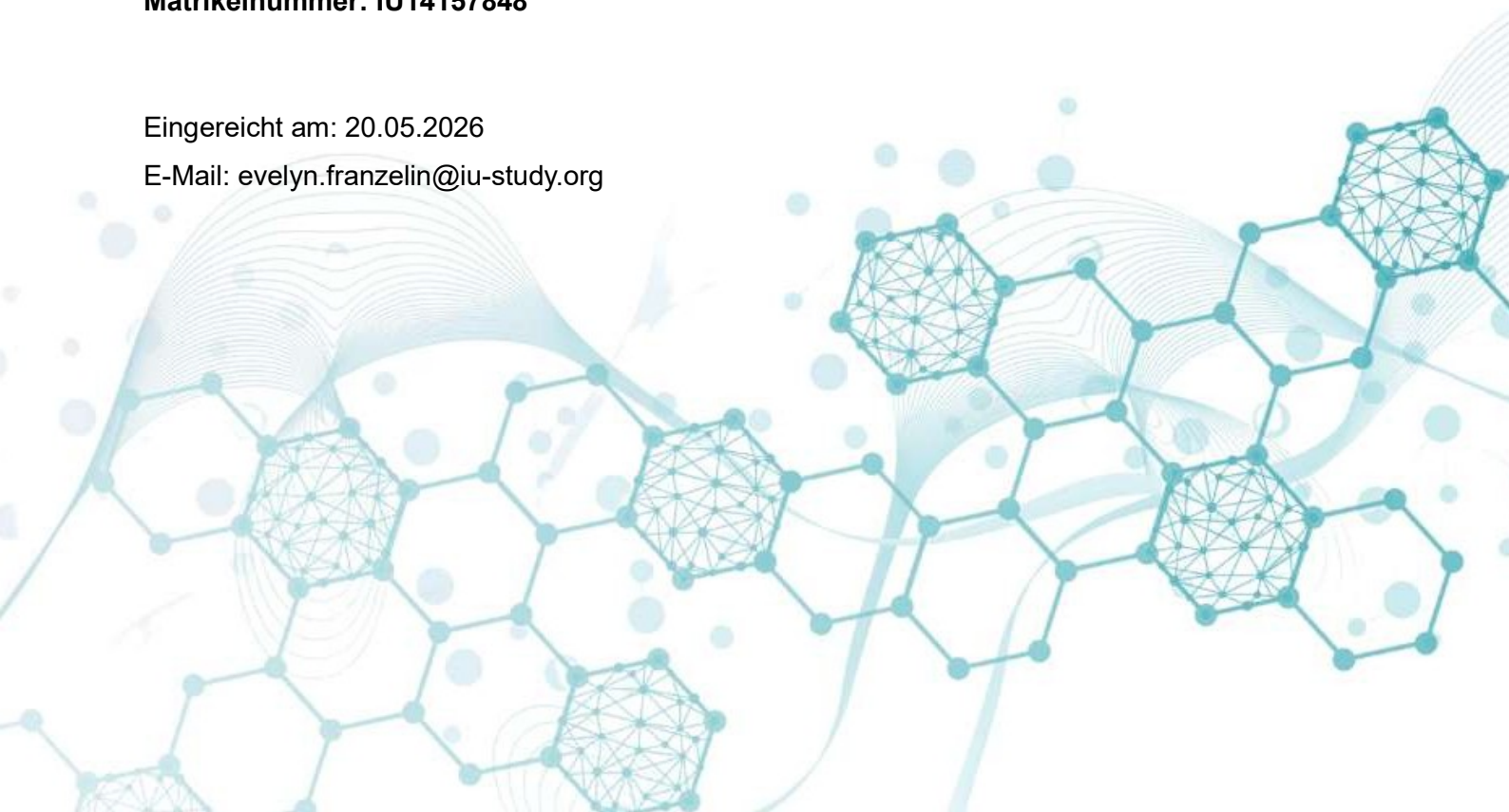
Tutor: Dr. Aiko Walte

Eingereicht von: Evelyn Franzelin

Matrikelnummer: IU14157848

Eingereicht am: 20.05.2026

E-Mail: evelyn.franzelin@iu-study.org



Inhaltsverzeichnis

Inhaltsverzeichnis	II
Abbildungsverzeichnis	II
Tabellenverzeichnis.....	II
1. Ausgangspunkt der Erarbeitungs- und Reflexionsphase	1
2. Untersuchung objektorientierter Konzepte in Python.....	1
3. Überarbeitung des Entity-Modells aus Phase 1	2
4. Gesamtarchitektur des Prototyps	3
5. Reflexion und Ausblick auf Phase 3	5
Literaturverzeichnis.....	6

Abbildungsverzeichnis

Abb. 1: Gesamtarchitektur des Studien-Dashboard-Prototyps	4
---	---

Tabellenverzeichnis

Tab. 1: Überarbeitung des Entity-Modells nach Feedback.....	3
---	---

1. Ausgangspunkt der Erarbeitungs- und Reflexionsphase

Die Konzeptionsphase des Studien-Dashboards wurde eingereicht und durch das Tutorfeedback grundsätzlich bestätigt. Positiv bewertet wurden die klar definierten Ziele, die übersichtliche Dashboard-Skizze und die sinnvolle Grundmodellierung. Gleichzeitig wurden mehrere Punkte genannt, die in der Erarbeitungs- und Reflexionsphase präzisiert werden müssen.

Im Mittelpunkt dieser Phase steht daher die Frage, wie das in Phase 1 entwickelte Modell sinnvoll in Python umgesetzt und zu einer tragfähigen Gesamtarchitektur erweitert werden kann. Dazu werden feste Wertebereiche als Enumerationen modelliert, Beziehungen zwischen den Klassen fachlich geschärft und Methoden wie `istAbgeschlossen()`, `istAbgegeben()` und `istBewertet()` als abgeleitete Prüfmethode eingeordnet.

Ziel dieser Phase ist nicht die vollständige Umsetzung des Prototyps, sondern die technische und fachliche Vorbereitung von Phase 3. Die Architektur soll Benutzeroberfläche, Fachlogik und dateibasierte Speicherung klar trennen, damit der spätere Code nachvollziehbar, wartbar und mit dem UML-Diagramm konsistent bleibt.

2. Untersuchung objektorientierter Konzepte in Python

Für die Umsetzung des Studien-Dashboards wurden zentrale objektorientierte Konzepte in Python betrachtet. Im Vordergrund steht dabei nicht die vollständige Programmierung des Prototyps, sondern die Frage, wie das in Phase 1 entwickelte Modell technisch sinnvoll abgebildet werden kann.

Die fachlichen Klassen Studium, Modul, Prüfungsleistung und Note können in Python als Klassen umgesetzt werden. Da diese Klassen vor allem Daten strukturieren und nur begrenzte eigene Logik enthalten, bietet sich die Verwendung von `dataclasses` an (Python Software Foundation, n.d.-a). Dadurch können Attribute übersichtlich definiert werden, ohne dass für jede Klasse ein ausführlicher Konstruktor manuell geschrieben werden muss. Für das Dashboard ist das sinnvoll, weil die Klassen gut lesbar bleiben und der Zusammenhang zwischen UML-Modell und Python-Code leichter nachvollziehbar ist.

Ein wichtiger Anpassungspunkt aus dem Feedback zur Konzeptionsphase betrifft die Verwendung von festen Wertebereichen. Attribute wie `studienmodell`, `status`, `subjektiveSchwierigkeit` und `pruefungsart` sollen nicht als freie Zeichenketten gespeichert werden. Stattdessen werden sie als Enums modelliert (Python Software Foundation, n.d.-b). Dadurch wird verhindert, dass unterschiedliche Schreibweisen oder ungültige Werte entstehen. Ein Modulstatus kann dann zum Beispiel nur einen der vorgesehenen Werte wie `NICHT_GESTARTET`, `IN_BEARBEITUNG`, `ABGEGEBEN`, `BESTANDEN` oder `PAUSIERT` annehmen.

Die folgende Erprobung zeigt beispielhaft, wie ein Modulstatus als Enum und eine abgeleitete Prüfmethode in Python umgesetzt werden können:

In JupyterLab wurde dazu ein kurzer Test mit einer Enum-Klasse `ModulStatus` und einer `dataclass` `Modul` durchgeführt. Dabei wurde geprüft, ob ein Modul nur vordefinierte Statuswerte wie `IN_BEARBEITUNG` oder `BESTANDEN` annehmen kann. Zusätzlich wurde getestet, ob die Methode `ist_abgeschlossen()` den Abschlusszustand aus dem vorhandenen Status ableitet. Der Test zeigte, dass kein zusätzlicher Zustand gespeichert werden muss, da die Methode lediglich den bestehenden Statuswert auswertet.

Die Beziehungen zwischen den Klassen lassen sich in Python über Attribute und Listen umsetzen. Ein Studium enthält mehrere Module, daher kann die Klasse `Studium` eine Liste von `Modul`-Objekten besitzen. Ein Modul enthält wiederum eine `Pruefungsleistung`. Diese Beziehung wird im weiteren Entwurf als starke Teil-Ganzes-Beziehung verstanden, weil eine Prüfungsleistung im Dashboard fachlich immer zu einem konkreten Modul gehört. Eine Note ist dagegen optional, da eine Prüfungsleistung bereits abgegeben sein kann, bevor eine Bewertung vorliegt.

Auch die Methoden aus dem UML-Modell wurden geprüft. Methoden wie `istAbgeschlossen()`, `istAbgegeben()` oder `istBewertet()` sollen keinen eigenen Zustand speichern. Sie dienen nur dazu, vorhandene Attribute auszuwerten. So kann `istAbgeschlossen()` beispielsweise prüfen, ob der Modulstatus `BESTANDEN` lautet. Die Methode `istBewertet()` kann prüfen, ob zu einer Prüfungsleistung bereits eine Note vorhanden ist. Dadurch bleibt der gespeicherte Zustand eindeutig, während fachliche Prüfregeln trotzdem direkt an der passenden Klasse gekapselt werden.

Für die spätere Umsetzung sollen Fachklassen, Oberfläche und Datenspeicherung getrennt bleiben. Berechnungen werden in Service-Klassen ausgelagert, während das Speichern und Laden der Daten über eine `Repository`-Klasse erfolgt.

3. Überarbeitung des Entity-Modells aus Phase 1

Auf Grundlage des Feedbacks zur Konzeptionsphase wurde das Entity-Modell aus Phase 1 überprüft und präzisiert. Die Grundstruktur mit den Klassen `Studium`, `Modul`, `Pruefungsleistung` und `Note` bleibt bestehen. Angepasst werden vor allem feste Wertebereiche, Beziehungen zwischen Klassen und die Begründung abgeleiteter Methoden.

Ein wichtiger Punkt betrifft Attribute mit festen Wertebereichen. In Phase 1 wurden mehrere dieser Attribute noch als `String` modelliert. Für die weitere Umsetzung ist es sinnvoller, diese Werte als Enumerationen abzubilden. Das betrifft insbesondere `studienmodell`, `status`, `subjektiveSchwierigkeit` und `pruefungsart`. Dadurch wird die Modellierung eindeutiger, da nur vorher festgelegte Werte verwendet werden können. Gleichzeitig wird die spätere Implementierung robuster, weil Tippfehler oder unterschiedliche Schreibweisen vermieden werden.

Auch die Beziehung zwischen den Klassen wurde erneut geprüft. Zwischen `Studium` und `Modul` wird im weiteren Entwurf eine Teil-Ganzes-Beziehung angenommen, da die Module im Dashboard als Bestandteile des persönlichen Studienverlaufs betrachtet werden. Besonders deutlich ist diese Abhängigkeit zwischen `Modul` und `Pruefungsleistung`. Eine Prüfungsleistung wird im Dashboard immer

einem konkreten Modul zugeordnet und hat ohne dieses Modul keinen eigenständigen fachlichen Kontext. Deshalb wird diese Beziehung als Komposition verstanden. Die Beziehung zwischen Prüfungsleistung und Note bleibt optional, weil eine Prüfungsleistung bereits existieren kann, bevor eine Bewertung vorliegt.

Die Methoden `istAbgeschlossen()`, `istAbgegeben()` und `istBewertet()` werden im überarbeiteten Modell nicht als zusätzliche Zustände verstanden. Sie dienen dazu, vorhandene Attribute auszuwerten. So kann `istAbgeschlossen()` aus dem Modulstatus abgeleitet werden, während `istBewertet()` prüft, ob eine Note vorhanden ist. Dadurch bleibt der gespeicherte Zustand eindeutig, während häufig benötigte fachliche Prüfungen trotzdem direkt über Methoden abgebildet werden können.

Das Attribut `schwierigkeit` wird zu `subjektiveSchwierigkeit` präzisiert, da es keine objektive Moduleigenschaft beschreibt, sondern eine persönliche Einschätzung für Planung und Priorisierung.

Tab. 1: Überarbeitung des Entity-Modells nach Feedback

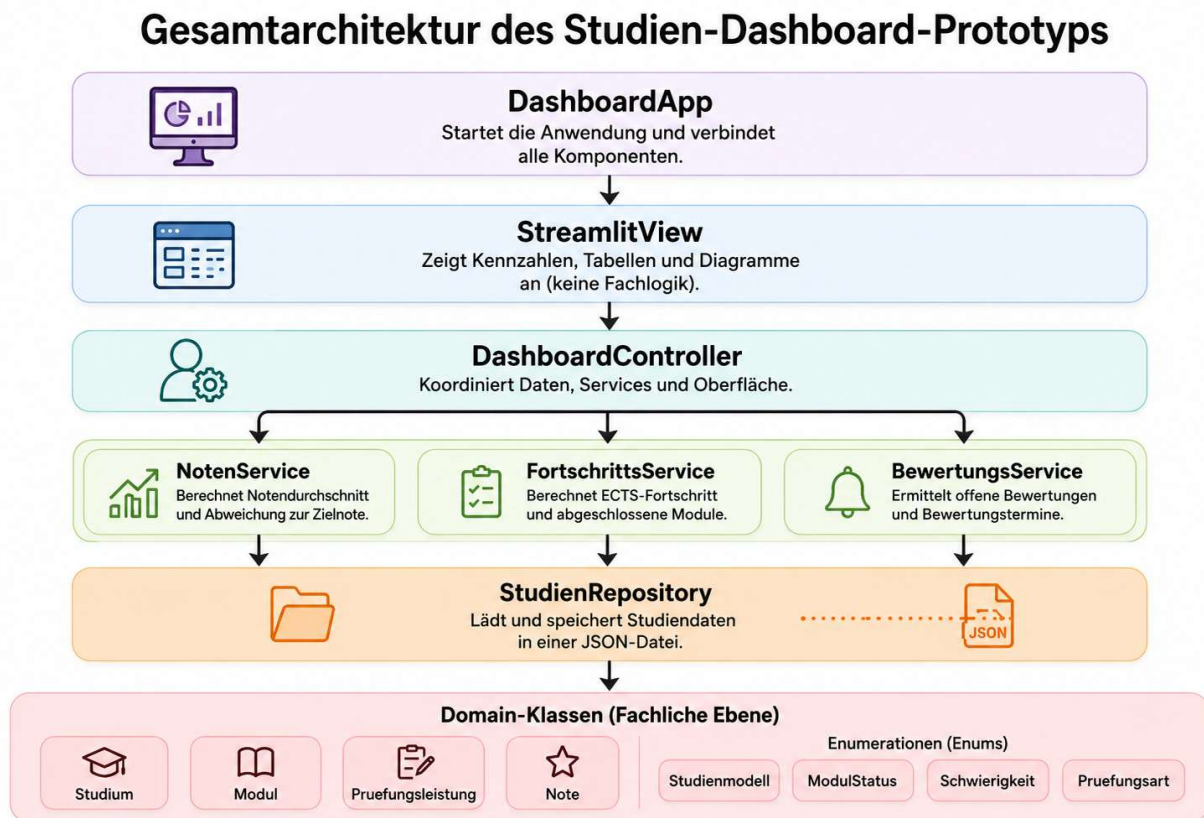
Feedbackpunkt	Anpassung in Phase 2	Begründung
Feste Werte wurden als String modelliert	Einführung von Enums für Studienmodell, ModulStatus, Schwierigkeit und Prüfungsart	Nur gültige Werte sollen auswählbar sein; unterschiedliche Schreibweisen werden vermieden.
Beziehungen sollten kritisch geprüft werden	Teil-Ganzes-Beziehungen werden stärker modelliert, besonders zwischen Modul und Prüfungsleistung	Eine Prüfungsleistung gehört fachlich zu einem konkreten Modul.
Status und Methoden wirkten teilweise redundant	Methoden werden als abgeleitete Prüfmethode verstanden	Die Methoden speichern keinen eigenen Zustand, sondern werten vorhandene Attribute aus.
Schwierigkeit als Moduleigenschaft unklar	Umbenennung zu <code>subjektiveSchwierigkeit</code>	Es handelt sich um eine persönliche Einschätzung, nicht um eine objektive Moduleigenschaft.

Quelle: Eigene Darstellung.

4. Gesamtarchitektur des Prototyps

Auf Grundlage der vorherigen Untersuchung wird für den Prototyp des Studien-Dashboards eine einfache Schichtenarchitektur vorgesehen. Ziel dieser Struktur ist es, die Benutzeroberfläche, die fachliche Logik, die Datenspeicherung und die fachlichen Klassen klar voneinander zu trennen. Dadurch soll vermieden werden, dass Berechnungen oder Speicheroperationen direkt in der Oberfläche umgesetzt werden.

Abb. 1: Gesamtarchitektur des Studien-Dashboard-Prototyps



Quelle: Eigene Darstellung, erstellt mit Canva.

Die Domain-Schicht enthält die fachlichen Klassen Studium, Modul, Pruefungsleistung und Note sowie die benötigten Enumerationen. Diese Klassen bilden den Studienverlauf und die zugehörigen Prüfungsinformationen ab. Sie enthalten nur fachlich passende Attribute und einfache abgeleitete Methoden, jedoch keine Logik zur Darstellung oder Speicherung.

Die Repository-Schicht übernimmt das Laden und Speichern der Studiendaten. Dafür ist die Klasse StudienRepository vorgesehen. Sie kapselt den Zugriff auf die JSON-Datei, sodass andere Programmteile nicht direkt mit der Datei arbeiten müssen (Python Software Foundation, n.d.-c). Wenn sich die Speicherform später ändern würde, müsste vor allem diese Schicht angepasst werden.

Die Service-Schicht enthält die fachliche Auswertungslogik. Der NotenService berechnet den aktuellen Notendurchschnitt und die Abweichung zur Zielnote. Der FortschrittsService berechnet erreichte ECTS, abgeschlossene Module und den Fortschritt im Verhältnis zum Teilzeit-II-Plan. Der BewertungsService ermittelt offene Bewertungen und relevante Bewertungstermine. Dadurch bleiben Berechnungen von der Oberfläche getrennt.

Der DashboardController koordiniert den Ablauf zwischen Repository, Services und Oberfläche. Er lädt die benötigten Daten, ruft die passenden Services auf und stellt die Ergebnisse gebündelt für die Darstellung bereit. Die StreamlitView ist nur für die Anzeige zuständig (Streamlit, n.d.). Sie stellt

Kennzahlen, Modulstatus, offene Bewertungen und einfache Visualisierungen dar, enthält aber keine eigene Fachlogik.

Die Klasse DashboardApp bildet den Startpunkt der Anwendung. Dort werden Repository, Services, Controller und View erzeugt und miteinander verbunden. Diese klare Aufteilung soll in Phase 3 helfen, den Code nachvollziehbar umzusetzen und die Übereinstimmung zwischen Architekturdigramm und tatsächlicher Implementierung zu sichern.

5. Reflexion und Ausblick auf Phase 3

Die Erarbeitungs- und Reflexionsphase zeigt, dass das in Phase 1 entwickelte Grundmodell grundsätzlich geeignet ist, aber für die Umsetzung in Python präzisiert werden muss. Besonders wichtig ist die Einführung von Enumerationen für feste Wertebereiche. Dadurch werden Attribute wie Studienmodell, Modulstatus, subjektive Schwierigkeit und Prüfungsart eindeutiger modelliert und spätere Eingabefehler reduziert.

Auch die Beziehungen zwischen den Klassen wurden geschärft. Die Prüfungsleistung wird im weiteren Entwurf enger an das Modul gebunden, da sie im Dashboard fachlich immer zu einem konkreten Modul gehört. Die Note bleibt dagegen optional, weil eine Prüfungsleistung bereits abgegeben sein kann, bevor eine Bewertung vorliegt. Diese Modellierung passt zum Ziel des Dashboards, offene Bewertungen sichtbar zu machen.

Die Methoden `istAbgeschlossen()`, `istAbgegeben()` und `istBewertet()` werden in der weiteren Umsetzung als abgeleitete Prüfmethode verstanden. Sie speichern keinen zusätzlichen Zustand, sondern werten vorhandene Attribute aus. Dadurch bleibt das Modell übersichtlich, ohne auf hilfreiche fachliche Methoden verzichten zu müssen.

Für Phase 3 ist besonders wichtig, dass die geplante Architektur auch im Code eingehalten wird. Die Oberfläche mit Streamlit soll nur für die Darstellung zuständig sein (Streamlit, n.d.). Berechnungen werden in Service-Klassen ausgelagert, während das StudienRepository das Laden und Speichern der JSON-Datei übernimmt. Der DashboardController verbindet diese Komponenten miteinander. So soll verhindert werden, dass Benutzeroberfläche, Fachlogik und Datenspeicherung zu stark vermischt werden.

In Phase 3 wird der Prototyp entlang dieser Architektur umgesetzt. Dabei muss darauf geachtet werden, dass das UML-Diagramm und der tatsächliche Code übereinstimmen. Klassen oder Methoden, die im Diagramm dargestellt werden, sollen daher auch im Code nachvollziehbar umgesetzt werden. Falls während der Implementierung Änderungen notwendig werden, müssen diese bewusst begründet und in der finalen Dokumentation berücksichtigt werden.

Literaturverzeichnis

Python Software Foundation. (n.d.-a). *dataclasses* — *Data Classes*. Python documentation.

<https://docs.python.org/3/library/dataclasses.html>

Python Software Foundation. (n.d.-b). *enum* — *Support for enumerations*. Python documentation.

<https://docs.python.org/3/library/enum.html>

Python Software Foundation. (n.d.-c). *json* — *JSON encoder and decoder*. Python documentation.

<https://docs.python.org/3/library/json.html>

Streamlit. (n.d.). *Streamlit documentation*. <https://docs.streamlit.io/>

Hinweis zur KI-Nutzung: Ein KI-Chatbot wurde punktuell zur sprachlichen Überarbeitung und zur Prüfung der Verständlichkeit einzelner Textstellen genutzt. Die Konzeption, fachliche Bewertung und finale Ausarbeitung erfolgten eigenständig.